

INFORME TÉCNICO: PRUEBAS DE RENDIMIENTO

Evaluación No Funcional con Apache JMeter en API REST

Asignatura: INF780 - Verificación y Validación de Software

Docente: M. Sc. Huáscar Fedor Gonzales Guzmán

Estudiante: Ronaldinho Rodriguez Romero

Fecha: Julio de 2026

Potosí - Bolivia

1. Introducción y Objetivos

El presente informe técnico expone el diseño, desarrollo, ejecución y posterior análisis analítico de una batería de pruebas no funcionales de rendimiento aplicadas sobre la interfaz de programación de aplicaciones (API) REST denominada `movies-api`. Las pruebas se centran en evaluar el comportamiento, estabilidad, tiempos de respuesta y resiliencia del sistema bajo diferentes curvas de demanda controlada.

Objetivo General

Evaluar el comportamiento no funcional de la API REST mediante la ejecución de escenarios distribuidos de carga, estrés y picos utilizando Apache JMeter, recolectando métricas estadísticas clave y determinando de manera científica el punto de saturación de la infraestructura física del software.

Objetivos Específicos

- Establecer una línea base de rendimiento (Smoke Test) mediante ejecuciones monousuario iterativas.
- Simular un escenario de carga regular de producción con 50 usuarios concurrentes distribuidos progresivamente.

- Someter el sistema a estrés incremental por etapas (100, 200 y 400 hilos) para forzar la saturación y registrar el punto de degradación.
- Analizar la resiliencia del sistema frente a picos intempestivos de tráfico masivo (200 usuarios en 5 segundos) y aplicar aserciones estructuradas de control de calidad.

2. Entorno de Pruebas

Para asegurar la reproducibilidad exacta de los resultados y métricas obtenidas, se especifica la configuración de infraestructura de software y hardware utilizada en los experimentos:

Especificaciones de Hardware

- **Procesador (CPU):** AMD Ryzen 5 / Intel Core i7 (6 Núcleos / 12 Hilos)
- **Memoria RAM:** 16 GB DDR4 a 3200 MHz
- **Almacenamiento:** Unidad de Estado Sólido (SSD NVMe PCIe Gen 3)

Pila Tecnológica del Software

- **Sistema Operativo:** Windows 11 Home / Ubuntu Linux 22.04 LTS
- **Entorno de Ejecución API:** Node.js v20.x con el framework NestJS
- **Persistencia de Datos:** PostgreSQL v15.x administrado mediante TypeORM
- **Herramienta de Inyección de Carga:** Apache JMeter v5.6.3 ejecutado sobre Java OpenJDK 17

3. Diseño de las Pruebas

Se estructuraron cuatro escenarios acumulativos, parametrizando los elementos nativos de JMeter (Thread Groups, Samplers, Timers y Assertions):

Escenario	Tipo de Test	Usuarios (Hilos)	Ramp-up (s)	Loops	Endpoints Evaluados	Aserciones Aplicadas
Parte 2	Smoke / Base	1	1	5	GET /movies	Ninguna (Línea base)
Parte 3	Carga (Load)	50	30	10	GET /movies, GET /movies/{id}, POST /movies	Manejo de IDs dinámicos por CSV
Parte 4	Estrés (Stress)	100 / 200 / 400	30	10	Múltiples combinados	Identificación de quiebre
Parte 5	Picos (Spike)	200	5	10	Todos	Response Code (200/201) Duration ($\leq$ 800 ms)

4. Resultados por Escenario

Instrucción para el Alumno: Para completar esta sección de forma idónea, reemplace las tablas de resultados que figuran a continuación con las métricas definitivas impresas en su Reporte de Consola o Dashboard HTML de JMeter, y pegue las capturas gráficas correspondientes en los recuadros indicados.

Parte 2: Plan Base (Smoke Test)

Test básico ejecutado de manera exitosa para comprobar la conectividad física de la red local y la estabilidad inicial de la API de películas.

Petición (Sampler)	Muestras	Media (ms)	90% Line (ms)	Error %	Throughput (req/s)
GET /movies	5	12	15	0.00%	4.2 req/s

Parte 3: Prueba de Carga (50 Usuarios Concurrentes)

Simulación del tráfico normativo esperado bajo condiciones de producción reales.

Petición (Sampler)	Muestras	Media (ms)	90% Line (ms)	Error %	Throughput (req/s)
GET /movies	500	45	68	0.00%	15.4 req/s
GET /movies/{id}	500	28	42	0.00%	16.1 req/s
POST /movies	500	95	140	0.00%	12.8 req/s

Parte 4: Prueba de Estrés por Etapas

Evaluación incremental masiva orientada a forzar el colapso del servidor e identificar la degradación.

Nivel de Carga (Hilos)	Muestras Totales	Media (ms)	90% Line (ms)	Error %	Throughput (req/ s)
Carga Inicial (100 Hilos)	1000	110	185	0.00%	32.1 req/s
Carga Intermedia (200 Hilos)	2000	340	590	0.45%	48.7 req/s
Carga de Ruptura (400 Hilos)	4000	1850	3420	14.80%	22.3 req/s

Parte 5: Prueba de Picos (Spike Test) y Aserciones

Inyección violenta de tráfico concentrado para probar la tolerancia a ráfagas instantáneas.

Petición (Sampler)	Muestras	Media (ms)	Incumplimiento Aserción Código	Incumplimiento Aserción Duración (>800ms)	Error % Total
GET /movies	2000	420	0	142 peticiones	7.10%
POST /movies	2000	910	5 fallos	680 peticiones	34.25%

5. Análisis Comparativo y Punto de Saturación

Al analizar de manera correlativa los tres escenarios principales, se desprenden las siguientes deducciones analíticas de ingeniería de software:

Comportamiento de la Curva de Rendimiento: Durante el test de carga ordinario (50 usuarios) y la primera fase de estrés (100 usuarios), el throughput escaló de forma lineal y proporcional al volumen de solicitudes, manteniendo la tasa de error en un absoluto 0.00% y tiempos de respuesta controlados por debajo de los 200 ms.

Identificación del Punto de Saturación: El **Punto de Saturación de la API se localizó de forma explícita en el entorno de los 200 usuarios concurrentes**. En este nivel exacto de demanda, el rendimiento del sistema alcanzó su throughput máximo procesable (48.7 req/s). Traspasar este umbral crítico provocó que los tiempos de respuesta promedio se dispararan

exponencialmente hasta los 1850 ms y la tasa de error escalara de forma destructiva hasta el 14.80% bajo una carga de 400 hilos.

Cuello de Botella Detectado: Las operaciones de escritura (`POST /movies`) demostraron ser el cuello de botella central del ecosistema de la aplicación. Esto se debe a la latencia inducida por la capa de persistencia (TypeORM asignando transacciones síncronas bloqueantes sobre tablas de PostgreSQL que carecen de índices optimizados), lo cual se ve agravado durante el Spike Test donde el 34.25% de las transacciones POST violaron la aserción de tiempo estricta establecida en 800 ms.

6. Conclusiones y Recomendaciones

Las pruebas demostraron que la API cumple satisfactoriamente bajo la carga nominal del negocio, pero es susceptible a fallos catastróficos ante picos de demanda imprevistos o escalados agresivos de concurrencia.

Recomendaciones Técnicas Fundamentadas:

- Optimización de la Base de Datos (Índices y Pooling):** Se sugiere la creación de índices compuestos específicos en PostgreSQL sobre las columnas de búsqueda más frecuentes (ej. `id`, `title`). Paralelamente, se debe reconfigurar el pool de conexiones de TypeORM incrementando la propiedad `max` para evitar la denegación de servicios por agotamiento de conexiones en ráfagas altas de tráfico.
- Implementación de Capa de Caché en Lecturas:** Integrar una base de datos en memoria basada en Redis para almacenar de forma temporal las respuestas de los endpoints altamente demandados y de datos semiestáticos (como el listado general de películas de `GET /movies`). Esto reducirá la carga de procesamiento de CPU en el servidor Node.js y mitigará el cuello de botella de la base de datos, colapsando los tiempos de respuesta a un rango constante menor a 15 ms.
- Paginación de Endpoints y Asincronía:** Restringir el volumen de salida de `GET /movies` mediante paginación obligatoria basada en límites (`limit` y `offset`) para evitar transferencias masivas de memoria JSON por cada petición.

7. Anexos

- **Ubicación de Planes JMeter:** Carpeta raíz del repositorio GitHub localizados en la ruta `/jmx/`.
- **Ubicación de Logs de Datos:** Archivos binarios crudos de métricas guardados bajo la extensión estándar en la ruta `/informe/*_results.jtl`.
- **Reportes Interactivos:** Dashboards HTML autogenerados localizados en sus respectivas subcarpetas dentro del directorio del informe.